LAPORAN PRAKTIKUM ALGORITMA
DAN PEMROGRAMAN 2

MODUL 5

“REKURSIF”

Disusun Oleh :
Farel Tri Julian
109082500163

Dosen :
WAHYU ANDI SAPUTRA, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

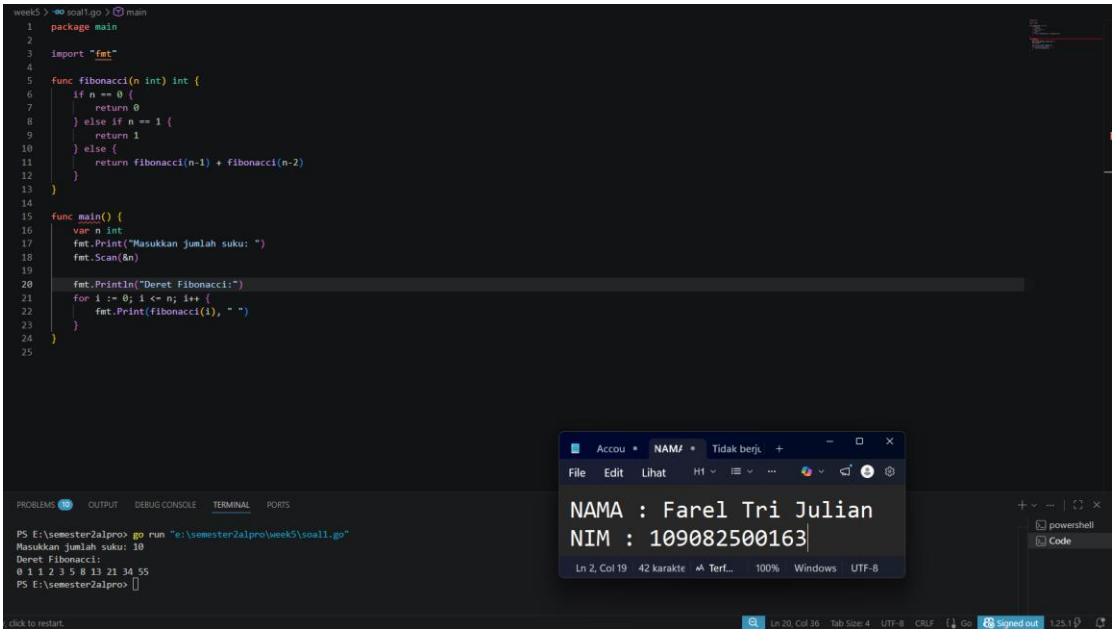
DASAR TEORI

Rekursif merupakan suatu metode pemrograman di mana sebuah fungsi memanggil dirinya sendiri untuk menyelesaikan suatu permasalahan yang lebih kecil dari masalah utama. Teknik rekursi biasanya digunakan untuk menyelesaikan masalah yang memiliki pola berulang dan dapat dipecah menjadi sub-masalah yang serupa. Dalam implementasinya, fungsi rekursif memiliki dua komponen penting, yaitu base case dan recursive case. Base case berfungsi sebagai kondisi penghentian agar pemanggilan fungsi tidak berlangsung tanpa batas, sedangkan recursive case adalah bagian di mana fungsi memanggil dirinya sendiri dengan parameter yang semakin mendekati kondisi akhir. Rekursi banyak digunakan dalam algoritma matematika seperti perhitungan faktorial, deret bilangan, pencarian, serta pembagian masalah kompleks menjadi bagian yang lebih sederhana. Namun, penggunaan rekursi juga harus diperhatikan karena dapat meningkatkan penggunaan memori dan menyebabkan kesalahan stack overflow apabila tidak memiliki batas yang jelas.

Tugas 1

```go
package main

import "fmt"

func fibonacci(n int) int {
    if n == 0 {
        return 0
    } else if n == 1 {
        return 1
    } else {
        return fibonacci(n-1) + fibonacci(n-2)
    }
}

func main() {
    var n int
    fmt.Print("Masukkan jumlah suku: ")
    fmt.Scan(&n)

    fmt.Println("Deret Fibonacci:")
    for i := 0; i <= n; i++ {
        fmt.Print(fibonacci(i), " ")
    }
}
```

Screenshots Output



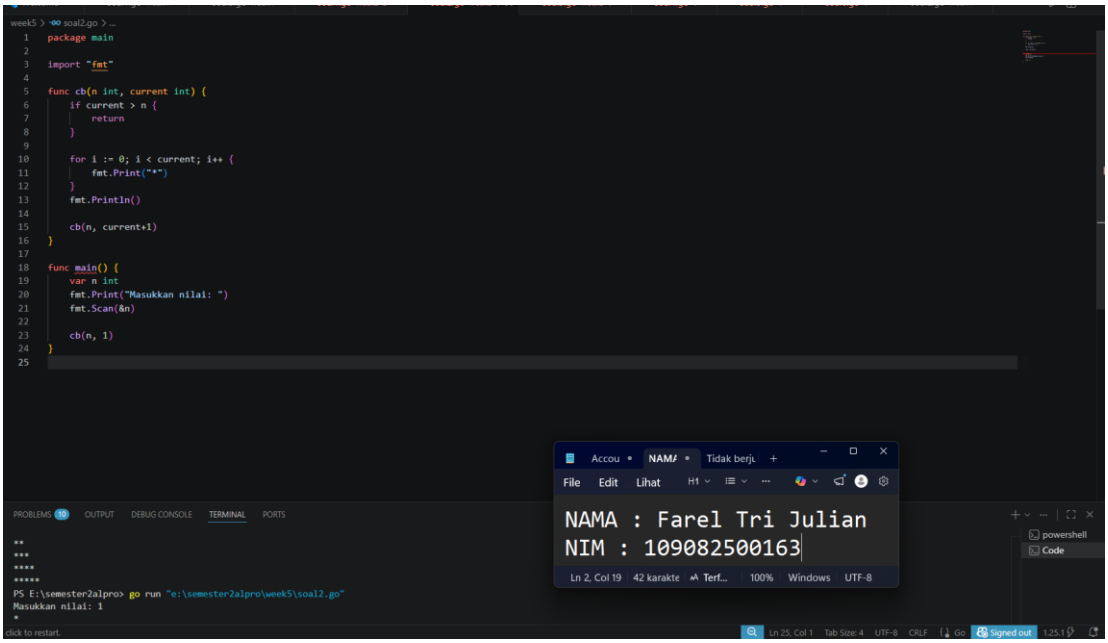
Deskripsi:

Program sederhana untuk menampilkan deret Fibonacci menggunakan bahasa Go. Program mendefinisikan fungsi rekursif Fibonacci yang menghitung nilai Fibonacci ke-n dengan memanggil dirinya sendiri, dengan kondisi dasar saat n bernilai 0 atau 1. Pada fungsi main, pengguna diminta memasukkan jumlah suku yang ingin ditampilkan, lalu program mencetak deret Fibonacci mulai dari suku ke-0 hingga suku ke-n dengan memanggil fungsi fibonacci untuk setiap indeks.

Tugas 2

```
package main

import "fmt"

func cb(n int, current int) {
    if current > n {
        return
    }

    for i := 0; i < current; i++ {
        fmt.Print("*")
    }
    fmt.Println()

    cb(n, current+1)
}

func main() {
    var n int
    fmt.Print("Masukkan nilai: ")
    fmt.Scan(&n)

    cb(n, 1)
}
```

Screenshots Output



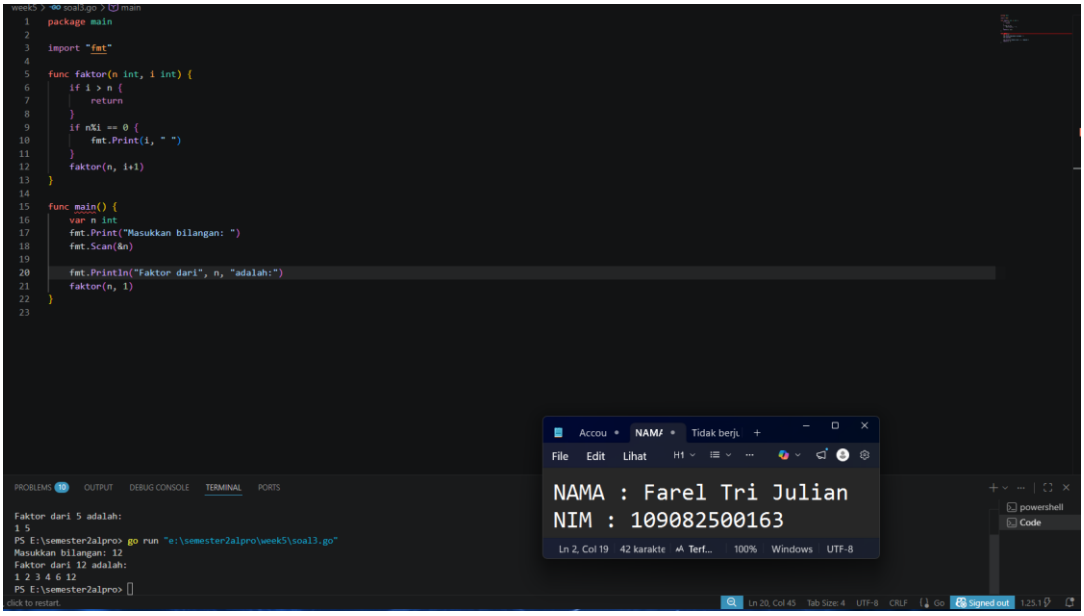
Deskripsi:

Program di atas adalah program Go yang mencetak pola segitiga bintang menggunakan pendekatan rekursif. Fungsi cb mencetak sejumlah bintang sesuai nilai current, lalu memanggil dirinya sendiri dengan current+1 hingga mencapai batas n. Pada fungsi main, pengguna diminta memasukkan nilai n sebagai tinggi segitiga, kemudian fungsi rekursif dipanggil mulai dari baris pertama untuk mencetak pola bertingkat dari satu bintang hingga n bintang.

Tugas 3

```go
package main

import "fmt"

func faktor(n int, i int) {
    if i > n {
        return
    }
    if n%i == 0 {
        fmt.Print(i, " ")
    }
    faktor(n, i+1)
}

func main() {
    var n int
    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&n)

    fmt.Println("Faktor dari", n, "adalah:")
    faktor(n, 1)
}
```

Screenshots Output



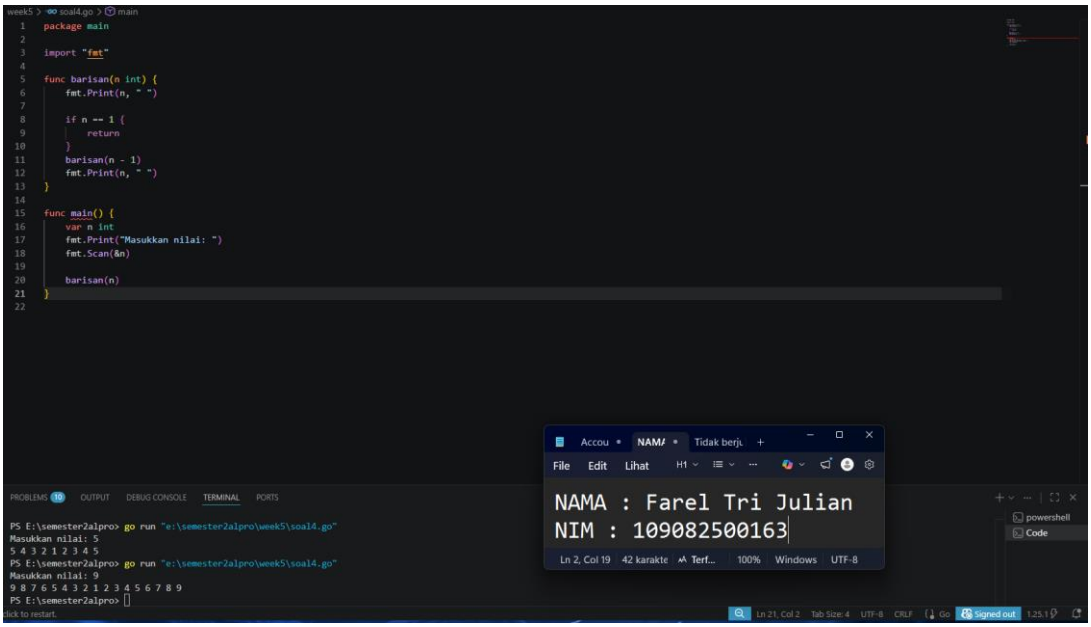
Deskripsi:

Program ini menampilkan semua faktor dari sebuah bilangan menggunakan fungsi rekursif. Fungsi faktor memeriksa apakah i adalah faktor dari n dengan mengecek apakah n habis dibagi i, lalu mencetaknya jika benar. Fungsi tersebut kemudian memanggil dirinya sendiri dengan nilai i yang bertambah satu hingga mencapai n. Pada fungsi main, pengguna diminta memasukkan sebuah bilangan, lalu program menampilkan semua faktor dari bilangan tersebut mulai dari 1 hingga bilangan itu sendiri.

Tugas 4

```
package main

import "fmt"

func barisan(n int) {
    fmt.Print(n, " ")

    if n == 1 {
        return
    }
    barisan(n - 1)
    fmt.Print(n, " ")
}

func main() {
    var n int
    fmt.Print("Masukkan nilai: ")
    fmt.Scan(&n)

    barisan(n)
}
```

Screenshots Output



Deskripsi:

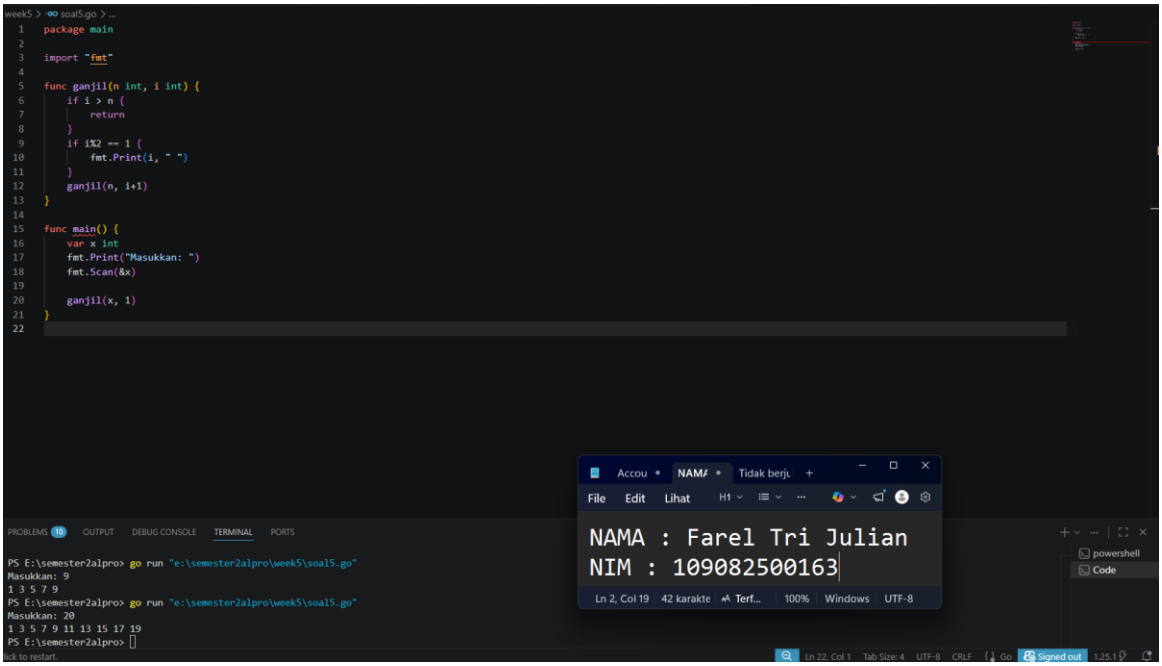
Program ini menampilkan pola barisan angka menggunakan rekursi. Fungsi barisan pertama-tama mencetak nilai n, lalu jika n belum mencapai 1, fungsi akan memanggil dirinya sendiri dengan nilai n-1. Setelah mencapai kondisi dasar, proses rekursi berhenti dan setiap nilai n dicetak kembali saat fungsi kembali, sehingga menghasilkan urutan menurun dari n ke 1 lalu kembali naik ke n.

Tugas 5

```
package main

import "fmt"

func ganjil(n int, i int) {
    if i > n {
        return
    }
    if i%2 == 1 {
        fmt.Print(i, " ")
    }
    ganjil(n, i+1)
}

func main() {
    var x int
    fmt.Print("Masukkan: ")
    fmt.Scan(&x)
```

```
    ganjil(x, 1)
}
```

Screenshots Output



Deskripsi:

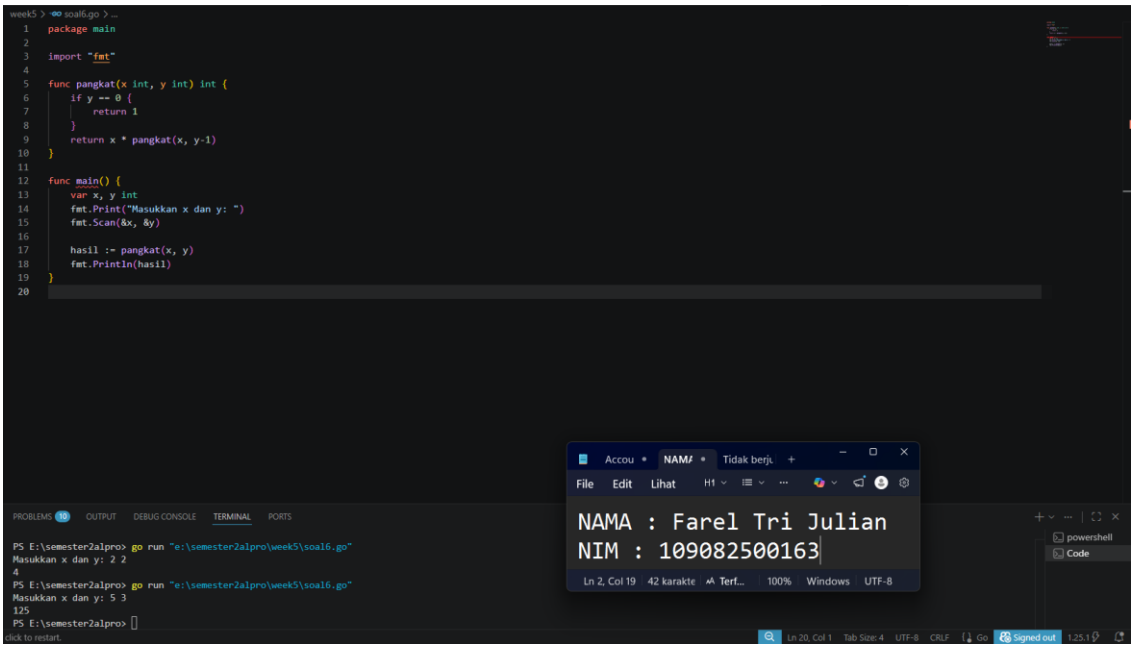
Program Go ini menampilkan deret bilangan ganjil dari 1 hingga batas yang ditentukan menggunakan rekursi. Fungsi ganjil memeriksa apakah nilai i ganjil dengan mengecek i % 2 == 1, lalu mencetaknya jika benar. Fungsi kemudian memanggil dirinya sendiri dengan i+1 hingga nilai i melebihi n. Pada fungsi main, pengguna diminta memasukkan nilai batas, kemudian program mencetak semua bilangan ganjil dari 1 sampai nilai tersebut.

Tugas 6

```
package main

import "fmt"

func pangkat(x int, y int) int {
    if y == 0 {
        return 1
    }
    return x * pangkat(x, y-1)
}
```

```
func main() {
    var x, y int
    fmt.Print("Masukkan x dan y: ")
    fmt.Scan(&x, &y)

    hasil := pangkat(x, y)
    fmt.Println(hasil)
}
```

Screenshots Output



Deskripsi:

Program ini menghitung nilai perpangkatan menggunakan fungsi rekursif. Fungsi pangkat(x, y) mengalikan nilai x dengan hasil pemanggilan dirinya sendiri hingga pangkat y mencapai 0, yang menjadi kondisi dasar dan mengembalikan nilai 1. Pada fungsi main, pengguna diminta memasukkan nilai x sebagai basis dan y sebagai pangkat, kemudian program menghitung dan menampilkan hasil dari x dipangkatkan y.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Downey, A. B. (2015). *Think Python: How to Think Like a Computer Scientist* (2nd ed.). Green Tea Press.